SPRING BOOT CRUD OPERATIONS

Complete Developer Documentation

Product Management System (PMS)

A Layer-by-Layer Guide to Building REST APIs

`Controller → Service → DAO → Repository → Model`

Table of Contents

1. What is CRUD? — The Big Picture

2. Project Architecture — The 5-Layer Pattern

3. The Flow of a Request — How It All Connects

4. Layer 1: Model — The Data Blueprint

5. Layer 2: Repository — The Database Bridge

6. Layer 3: DAO — The Data Access Object

7. Layer 4: Service — The Business Brain

8. Layer 5: Controller — The HTTP Gateway

9. CRUD Operations — Full Walkthrough

10. API Endpoints Summary

11. Key Annotations Reference

12. Common Mistakes & How to Avoid Them

13. Memory Tips — Never Forget CRUD Again

1. What is CRUD? — The Big Picture

CRUD stands for the four fundamental operations that any data-driven application performs. Every app you have ever used — Instagram, Amazon, your bank's website — runs on CRUD at its core.

Letter	Operation
C	CREATE — Add new data (e.g., register a user, add a product)
R	READ — Retrieve/fetch data (e.g., view a profile, list products)
U	UPDATE — Modify existing data (e.g., edit a product's price)
D	DELETE — Remove data (e.g., delete an account, remove a product)

> 🎯 **Why CRUD Matters**
>
> CRUD is not just a Spring Boot concept. It applies to every database-driven application in every programming language. Master CRUD in Spring Boot and you understand the backbone of web development.

In Spring Boot, CRUD maps directly to HTTP methods:

CRUD Operation	HTTP Method
CREATE	POST — Sends new data in the request body
READ (All)	GET — Fetches all records
READ (One)	GET with ID — Fetches a single record by ID
UPDATE	PUT — Replaces a record with new data
DELETE (One)	DELETE with ID — Removes one record

DELETE (All)	DELETE — Removes all records

2. Project Architecture — The 5-Layer Pattern

This project follows the standard Spring Boot layered architecture. Each layer has ONE specific job. This is called the Separation of Concerns principle — one of the most important concepts in software engineering.

Layer / File	Responsibility
Model (Product.java)	Defines what a Product looks like — its fields and structure
Repository (ProductRepository.java)	Speaks directly to the database using JPA/Hibernate
DAO (ProductDao.java)	Translates repository results into business-usable data
Service (ProductService.java)	Contains business logic; bridges Controller and DAO
Controller (ProductController.java)	Receives HTTP requests and returns HTTP responses

📐 **The Golden Rule of Layers**

A layer must NEVER skip a level. The Controller calls Service. The Service calls DAO. The DAO calls Repository. Never let the Controller talk directly to the Repository. This keeps your code clean, testable, and maintainable.

Think of it like a restaurant:

- **Customer (HTTP Request) talks to Waiter**
- **Waiter (Controller) takes the order to the Kitchen Manager**
- **Kitchen Manager (Service) tells the Cook what to make**
- **Cook (DAO) goes to the Pantry (Repository/Database) and gets the ingredients**

- **The dish (Response) comes back through the same chain**

Code:

Controller

```
package com.pms.controller;


import java.util.List;


import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.web.bind.annotation.DeleteMapping;

import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.PathVariable;

import org.springframework.web.bind.annotation.PostMapping;

import org.springframework.web.bind.annotation.PutMapping;

import org.springframework.web.bind.annotation.RequestBody;

import org.springframework.web.bind.annotation.RequestMapping;

import org.springframework.web.bind.annotation.RestController;


import com.pms.model.Product;

import com.pms.service.ProductService;


@RestController

@RequestMapping("/product")

public class ProductController {
```

```
@Autowired
ProductService service;

@GetMapping("/getData")
public List<Product> getData() {
    return service.getData();
}

@GetMapping("/getData/{id}")
public Product getDataById(@PathVariable Long id) {
    return service.getDataById(id);
}

@PostMapping("/insert")
public String insert(@RequestBody Product product) {
    return service.insert(product);
}

@PutMapping("/update/{id}")
public String update(@PathVariable Long id, @RequestBody Product product) {
    return service.update(id, product);
}
```

```
    @DeleteMapping("/delete")
    public String deleteAll() {
        return service.deleteAll();
    }

    @DeleteMapping("/delete/{id}")
    public String deleteById(@PathVariable Long id) {
        return service.deleteById(id);
    }
}
```

Service:

```
package com.pms.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.pms.dao.ProductDao;
import com.pms.model.Product;
```

```
@Service
public class ProductService {

    @Autowired
    ProductDao dao;

    public List<Product> getData() {
        return dao.getData();
    }

    public Product getDataById(Long id) {
        return dao.getDataById(id);
    }

    public String insert(Product product) {
        return dao.insert(product);
    }

    public String update(Long id, Product product) {
        return dao.update(id, product);
    }
```

```
    public String deleteAll() {
        return dao.deleteAll();
    }

    public String deleteById(Long id) {
        return dao.deleteById(id);
    }
}
```

Dao:

```
package com.pms.dao;

import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;

import com.pms.model.Product;
import com.pms.repository.ProductRepository;
```

```
@Repository
public class ProductDao {

    @Autowired
    ProductRepository repo;

    public List<Product> getData() {
        return repo.findAll();
    }

    public Product getDataById(Long id) {
        Optional<Product> optional = repo.findById(id);
        if (optional.isPresent()) {
            return optional.get();
        }
        return null;
    }

    public String insert(Product product) {
        repo.save(product);
        return "Product inserted successfully";
    }
```

```
public String update(Long id, Product product) {
    Optional<Product> optional = repo.findById(id);
    if (optional.isPresent()) {
        product.setId(id);
        repo.save(product);
        return "Product updated successfully";
    }
    return "Product not found with id: " + id;
}

public String deleteAll() {
    repo.deleteAll();
    return "All products deleted successfully";
}

public String deleteById(Long id) {
    if (repo.existsById(id)) {
        repo.deleteById(id);
        return "Product deleted successfully";
    }
    return "Product not found with id: " + id;
```

```
    }

}
```

Repository:

```
package com.pms.repository;


import org.hibernate.boot.models.JpaAnnotations;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;


import com.pms.model.Product;


@Repository

public interface ProductRepository extends  JpaRepository<Product, Long> {


}
```

Model:

```
package com.pms.model;


import jakarta.persistence.Entity;

import jakarta.persistence.GeneratedValue;

import jakarta.persistence.GenerationType;

import jakarta.persistence.Id;
```

```
@Entity

public class Product {


@Id

@GeneratedValue(strategy = GenerationType.IDENTITY)

private long id;


private String name;

private String Description;

private String Category;

private int price;

private int stockQuantity;

private boolean inStock;

public long getId() {

return id;

}

public void setId(long id) {

this.id = id;

}

public String getName() {

return name;
```

```
}
public void setName(String name) {
this.name = name;
}
public String getDescription() {
return Description;
}
public void setDescription(String description) {
Description = description;
}
public String getCategory() {
return Category;
}
public void setCategory(String category) {
Category = category;
}
public int getPrice() {
return price;
}
public void setPrice(int price) {
this.price = price;
}
```

```
public int getStockQuantity() {

return stockQuantity;

}

public void setStockQuantity(int stockQuantity) {

this.stockQuantity = stockQuantity;

}

public boolean isInStock() {

return inStock;

}

public void setInStock(boolean inStock) {

this.inStock = inStock;

}

}
```

3. The Flow of a Request — How It All Connects

Every HTTP request travels through all 5 layers and returns through the same path. Here is the complete journey of a request:

Step	What Happens
Step 1	Client sends HTTP request (e.g., GET /product/getData/5)
Step 2	Controller receives it — @GetMapping("/getData/{id}") is triggered
Step 3	Controller calls service.getDataById(5)
Step 4	Service calls dao.getDataById(5)
Step 5	DAO calls repo.findById(5) — actual DB query runs
Step 6	Database returns the Product record
Step 7	DAO wraps it and returns Product to Service
Step 8	Service returns Product to Controller
Step 9	Controller returns Product as JSON to the Client

```
HTTP GET /product/getData/5

  → ProductController.getDataById(5)

    → ProductService.getDataById(5)

      → ProductDao.getDataById(5)

        → ProductRepository.findById(5)  [SQL: SELECT * FROM product WHERE id=5]

          ← Returns Optional<Product>

        ← Returns Product

      ← Returns Product

    ← Returns Product

  ← Returns Product as JSON to client
```

4. Layer 1: Model — The Data Blueprint

The Model class (also called an Entity) is the blueprint of your data. It represents exactly one table in your database. Every field in the class becomes a column in the table.

File: com.pms.model.Product

Purpose: Defines the structure of a Product — what information a product holds.

Why it exists: Without a Model, the application has no idea what data to store or retrieve.

```
@Entity
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;

    private String name;
    private String Description;
    private String Category;
    private int price;
    private int stockQuantity;
    private boolean inStock;

    // ... getters and setters
}
```

Annotation Breakdown

Annotation	What It Does & Why It Is Used
@Entity	Tells Spring: 'This class maps to a database table.' Without this, JPA ignores the class entirely. Spring will auto-create a table named 'product' in your database.
@Id	Marks the primary key field. Every database table needs a unique identifier for each row. This tells JPA which field is that unique identifier.
@GeneratedValue(strategy = GenerationType.IDENTITY)	Tells the database to auto-increment the ID value. You never need to manually set the ID — the database handles it. IDENTITY means it uses the DB's auto-increment feature (e.g., 1, 2, 3...).

Field Breakdown

Field	Description
id (long)	Unique identifier for each product. Auto-generated. Never set this manually.
name (String)	Product name (e.g., 'Laptop', 'Keyboard')
Description (String)	Detailed description of the product
Category (String)	Product category (e.g., 'Electronics', 'Clothing')
price (int)	Price of the product in whole number units
stockQuantity (int)	How many units are currently in stock
inStock (boolean)	true if product is available; false if out of stock

> 💡 **Remember This Forever**
>
> The Model = The Database Table. One Java class = One DB table. One Java field = One DB column. @Entity is the magic annotation that makes this happen. If you forget @Entity, your class is just a plain Java class with no database power.

5. Layer 2: Repository — The Database Bridge

The Repository is your direct line to the database. It is an interface (not a class) that you simply extend. Spring Boot automatically provides all the database operations — you write zero SQL.

File: com.pms.repository.ProductRepository

Purpose: Provides all database CRUD operations automatically through JPA.

Why it exists: Without this, you would need to write SQL queries for every operation. JPA does it for you.

```
@Repository

public interface ProductRepository extends JpaRepository<Product, Long> {

    // Nothing needed here!

    // JpaRepository gives you all these methods for FREE:

    //

    // findAll()         — SELECT * FROM product

    // findById(id)      — SELECT * FROM product WHERE id=?

    // save(entity)      — INSERT or UPDATE

    // deleteById(id)    — DELETE FROM product WHERE id=?

    // deleteAll()       — DELETE FROM product

    // existsById(id)    — SELECT COUNT(*) WHERE id=?

}
```

Understanding JpaRepository<Product, Long>

JpaRepository takes two type parameters inside the angle brackets < >:

Parameter	Explanation
Product	The Entity class this repository manages. JpaRepository knows to work with the 'product' table.
Long	The data type of the primary key (the @Id field). Since our id is a long, we use Long here.

Free Methods You Get from JpaRepository

Method	What It Does
findAll()	Returns a List<Product> of ALL products in the database
findById(Long id)	Returns Optional<Product> — either the product or empty if not found
save(Product p)	Inserts if new, updates if ID already exists in the database
deleteById(Long id)	Deletes the product with the given ID
deleteAll()	Deletes every record in the product table
existsById(Long id)	Returns true if a product with that ID exists, false otherwise
count()	Returns the total number of products in the database

🧠 **Key Insight: Optional<T>**

findById() returns Optional<Product>, not Product directly. Optional is a wrapper that handles the case where the product might not exist. Use optional.isPresent() to check if a value exists, then optional.get() to retrieve it. This prevents NullPointerException — one of the most common bugs in Java.

6. Layer 3: DAO — The Data Access Object

DAO stands for Data Access Object. It sits between the Service layer and the Repository. Its job is to use the Repository to perform database operations and add logic like 'does this record exist before updating it?'

File: com.pms.dao.ProductDao

Purpose: Calls repository methods and handles data-level logic (like existence checks).

Why it exists: Keeps data access code separate from business logic (Service) and HTTP handling (Controller).

```java
@Repository
public class ProductDao {

    @Autowired
    ProductRepository repo;

    public List<Product> getData() {
        return repo.findAll();              // Gets all products
    }

    public Product getDataById(Long id) {
        Optional<Product> optional = repo.findById(id);
        if (optional.isPresent()) {
            return optional.get();           // Product found
        }
        return null;                         // Not found
    }

    public String insert(Product product) {
        repo.save(product);
        return "Product inserted successfully";
    }

    public String update(Long id, Product product) {
        Optional<Product> optional = repo.findById(id);
        if (optional.isPresent()) {
            product.setId(id);               // CRITICAL: set the ID!
            repo.save(product);
            return "Product updated successfully";
        }
        return "Product not found with id: " + id;
    }
```

Critical Logic in Each DAO Method

Method	Critical Logic Explained
getDataById()	Uses Optional<Product> from findById(). Always check isPresent() before calling get(). If you call get() on an empty Optional, it throws NoSuchElementException.
insert()	Calls repo.save(product). Since the product has no ID yet (id=0), JPA treats it as a new INSERT operation.
update()	MUST call product.setId(id) before save(). If you do not set the ID, JPA will INSERT a new record instead of UPDATE the existing one! This is the most common update bug.
deleteById()	Checks existsById() before deleting. Without this check, deleteById() on a non-existent ID would throw EmptyResultDataAccessException.

⚠️ Critical Update Rule

In the update() method, you MUST call product.setId(id) before calling repo.save(product). JPA uses the ID to decide INSERT vs UPDATE. No ID = new record inserted. ID set = existing record updated. Forgetting setId() is one of the most common bugs beginners make.

7. Layer 4: Service — The Business Brain

The Service layer is the brain of the application. It holds business logic and acts as the middleman between the Controller (HTTP layer) and the DAO (data layer). In this project, the Service is clean — it simply delegates to the DAO — but in real projects, this is where complex rules live.

File: com.pms.service.ProductService

Purpose: Contains business logic and coordinates between Controller and DAO.

Why it exists: Separates business rules from HTTP handling and database access. Makes code testable.

```
@Service
public class ProductService {

    @Autowired
    ProductDao dao;

    public List<Product> getData() {
        return dao.getData();
    }

    public Product getDataById(Long id) {
        return dao.getDataById(id);
    }

    public String insert(Product product) {
        return dao.insert(product);
    }

    public String update(Long id, Product product) {
        return dao.update(id, product);
    }

    public String deleteAll() {
        return dao.deleteAll();
    }

    public String deleteById(Long id) {
        return dao.deleteById(id);
    }
}
```

What Goes in the Service in Real Projects?

In this basic PMS project, the Service simply passes calls to the DAO. In a real-world application, the Service would contain rules like:

- **Validation: 'Price cannot be negative — throw error if price < 0'**
- **Business rules: 'If stock hits 0, automatically set inStock = false'**
- **Calling multiple DAOs: 'When creating an order, update both Order and Inventory'**
- **Sending emails or notifications after a product is created**
- **Authorization checks: 'Only admin users can delete products'**

> 💡 **Why Not Put Logic in the Controller?**
>
> Controllers should ONLY handle HTTP concerns (reading request, sending response). If you put business logic in the Controller, you cannot test it without making HTTP calls. Keeping logic in the Service means you can unit test business rules independently of HTTP.

8. Layer 5: Controller — The HTTP Gateway

The Controller is the entry point of your application. It listens for HTTP requests on specific URLs (called endpoints), processes them, and sends back HTTP responses. It is the only layer that the outside world (browser, mobile app, Postman) communicates with.

File: com.pms.controller.ProductController

Purpose: Receives HTTP requests, calls Service, returns HTTP responses.

Why it exists: The interface between the HTTP world and your Java application.

```
@RestController
@RequestMapping("/product")
public class ProductController {

    @Autowired
    ProductService service;

    @GetMapping("/getData")
    public List<Product> getData() {
        return service.getData();
    }

    @GetMapping("/getData/{id}")
    public Product getDataById(@PathVariable Long id) {
        return service.getDataById(id);
    }

    @PostMapping("/insert")
    public String insert(@RequestBody Product product) {
        return service.insert(product);
    }

    @PutMapping("/update/{id}")
    public String update(@PathVariable Long id, @RequestBody Product product) {
        return service.update(id, product);
    }

    @DeleteMapping("/delete")
    public String deleteAll() {
        return service.deleteAll();
    }
```

Controller Annotation Deep Dive

Annotation	Purpose & When to Use
@RestController	Combines @Controller + @ResponseBody. Tells Spring: 'Every method in this class should return data (JSON/text) directly in the HTTP response body, not a view/HTML page.'
@RequestMapping("/product")	Sets the base URL for ALL endpoints in this controller. Every endpoint URL starts with /product. Think of it as the 'folder' prefix.
@GetMapping("/getData")	Maps HTTP GET requests to /product/getData. Use GET for READ operations — fetching data.
@GetMapping("/getData/{id}")	Maps GET requests with an ID in the URL. The {id} is a path variable — a dynamic part of the URL.
@PostMapping("/insert")	Maps HTTP POST requests to /product/insert. Use POST for CREATE — sending new data.
@PutMapping("/update/{id}")	Maps HTTP PUT requests to /product/update/{id}. Use PUT for UPDATE — replacing existing data.
@DeleteMapping("/delete")	Maps HTTP DELETE requests to /product/delete. Deletes all records.
@DeleteMapping("/delete/{id}")	Maps DELETE requests with an ID. Deletes one specific record.
@PathVariable	Extracts the value from the URL path. In /getData/5, this pulls out the '5' and injects it as the id parameter.
@RequestBody	Tells Spring to read the HTTP request body (JSON) and automatically convert it into a Java object (Product). Used for POST and PUT.
@Autowired	Tells Spring to automatically inject the dependency. Spring creates the ProductService object and 'wires' it into this controller — you never use 'new ProductService()'.

9. CRUD Operations — Full Walkthrough

9.1 CREATE — Insert a New Product

Use Case: A store manager adds a new laptop to the inventory.

Detail	Value
HTTP Method	POST
Endpoint URL	http://localhost:8080/product/insert
Request Body	JSON with product details (no id required)
Response	String: 'Product inserted successfully'

Request Body (JSON you send):

```
{
    "name": "Gaming Laptop",
    "Description": "High performance laptop with RTX 4090",
    "Category": "Electronics",
    "price": 150000,
    "stockQuantity": 10,
    "inStock": true
}
```

What happens internally:

- **@RequestBody converts JSON → Product Java object**
- **Controller calls service.insert(product)**

- **Service calls dao.insert(product)**
- **DAO calls repo.save(product) — JPA sees id=0, runs INSERT SQL**
- **Database assigns id=1 (auto-increment), returns success**

9.2 READ ALL — Get All Products

Use Case: Display all products in the store's catalog.

Detail	Value
HTTP Method	GET
Endpoint URL	http://localhost:8080/product/getData
Request Body	None required
Response	JSON array of all products

Response (JSON returned):

```
[
    { "id": 1, "name": "Gaming Laptop", "price": 150000, "inStock": true },
    { "id": 2, "name": "Mechanical Keyboard", "price": 5000, "inStock": true }
]
```

9.3 READ ONE — Get Product by ID

Use Case: A user clicks on a specific product to see its details.

Detail	Value
HTTP Method	GET
Endpoint URL	http://localhost:8080/product/getData/1
Path Variable	1 (the product ID)
Response	Single Product JSON or null if not found

9.4 UPDATE — Modify an Existing Product

Use Case: The price of a laptop changes. Update it in the system.

Detail	Value
HTTP Method	PUT
Endpoint URL	http://localhost:8080/product/update/1
Path Variable	1 (ID of product to update)
Request Body	JSON with new product data
Response	String: 'Product updated successfully' or 'Product not found'

Request Body:

```
{
    "name": "Gaming Laptop Pro",
    "Description": "Updated model with RTX 5090",
    "Category": "Electronics",
    "price": 200000,
    "stockQuantity": 5,
    "inStock": true
}
```

> ⚠️ **Why setId() Is Critical in Update**
>
> In the DAO's update() method, product.setId(id) MUST be called before repo.save(product). The JSON body does not include the id field, so the Product object has id=0. If you call repo.save() with id=0, JPA thinks it is a new object and INSERTs a duplicate! Setting the ID first tells JPA to UPDATE the existing record.

9.5 DELETE ONE — Delete a Product by ID

Detail	Value
HTTP Method	DELETE
Endpoint URL	http://localhost:8080/product/delete/1
Path Variable	1 (ID of product to delete)
Response	String: 'Product deleted successfully' or 'Product not found'

9.6 DELETE ALL — Delete Every Product

Detail	Value
HTTP Method	DELETE
Endpoint URL	http://localhost:8080/product/delete
Request Body	None
Response	String: 'All products deleted successfully'

10. API Endpoints Summary

HTTP Method + URL	Operation
GET /product/getData	READ — Returns all products as JSON array
GET /product/getData/{id}	READ — Returns one product by ID
POST /product/insert	CREATE — Adds a new product (body required)
PUT /product/update/{id}	UPDATE — Updates product by ID (body required)
DELETE /product/delete	DELETE — Removes all products
DELETE /product/delete/{id}	DELETE — Removes one product by ID

11. Key Annotations Reference

Annotation	Meaning & Layer
@Entity	Model — Maps this class to a database table
@Id	Model — Marks the primary key field
@GeneratedValue(strategy=IDENTITY)	Model — Auto-increments the ID in the database
@Repository	Repository & DAO — Marks as data access component; enables exception translation
@Service	Service — Marks as business logic component
@RestController	Controller — Marks as REST API controller; returns JSON directly
@RequestMapping	Controller — Sets base URL path for all endpoints
@GetMapping	Controller — Maps HTTP GET requests to this method
@PostMapping	Controller — Maps HTTP POST requests to this method
@PutMapping	Controller — Maps HTTP PUT requests to this method
@DeleteMapping	Controller — Maps HTTP DELETE requests to this method
@Autowired	Any layer — Spring auto-injects the dependency
@PathVariable	Controller — Extracts value from URL path (e.g., /delete/{id})
@RequestBody	Controller — Converts JSON request body to Java object

12. Common Mistakes & How to Avoid Them

Mistake	Why It Happens & Fix
Forgot product.setId(id) in update()	Without setting the ID, JPA sees a new object and INSERTs instead of UPDATEs. Always call setId() before save() in update operations.
Calling optional.get() without isPresent()	If findById() returns empty Optional and you call get(), you get NoSuchElementException. Always check isPresent() first or use optional.orElse(null).
Using @Controller instead of @RestController	@Controller returns view names (HTML pages). @RestController returns data (JSON). For REST APIs, always use @RestController.
Skipping layers (Controller calls Repository directly)	This breaks separation of concerns. If business logic changes, you must change multiple places. Always go through Service → DAO → Repository.
Sending id in the POST request body	When inserting, never send the id in JSON. The database generates it automatically. Sending id=0 is fine; JPA ignores it.
Missing @RequestBody on POST/PUT methods	Without @RequestBody, Spring cannot map the JSON body to your Java object. The Product object will be null, causing NullPointerException.
Wrong HTTP method for an operation	Using GET for insert, or POST for delete. Match HTTP methods to CRUD: POST=Create, GET=Read, PUT=Update, DELETE=Delete. REST clients and tools depend on correct methods.

13. Memory Tips — Never Forget CRUD Again

🧠 **The CRUD-HTTP Mantra**

Create = POST (posting new things) | Read = GET (getting data) | Update = PUT (putting new data in) | Delete = DELETE (obvious!)

🏗️ **The Layer Mantra**

Model = Structure | Repository = Database | DAO = Access Logic | Service = Business Brain | Controller = HTTP Gateway

📌 **The 5 Rules to Memorize**

1. @Entity makes a class a DB table
2. JpaRepository<Entity, IDType> gives free CRUD methods
3. findById() returns Optional — always check isPresent()
4. In update(), always call setId() before save()
5. @RestController + @RequestMapping + HTTP mappings = your API

The Architecture in One Sentence

HTTP Request → @RestController receives it → calls @Service for business logic → @Service calls @Repository(DAO) for data → DAO uses JpaRepository to query the @Entity table in the database → response flows back up.

— End of Documentation —

CHAPTER: HIBERNATE & ORM

From JDBC Pain to Hibernate Power — A Complete Guide

14. What is ORM? — Object Relational Mapping

Databases store data in tables (rows and columns). Java stores data in objects (classes and fields). These two worlds speak different languages. ORM (Object Relational Mapping) is the translator between them.

Without ORM, a developer must manually write SQL queries, manually map every ResultSet column to a Java field, manually handle connections, and manually manage transactions. With ORM, you write Java code and the framework generates the SQL for you automatically.

🎯 **The Core Idea of ORM**

One Java class = One database table. One Java object = One row in that table. One Java field = One column in that table. ORM makes this mapping automatic — you think in Java objects, the framework thinks in SQL tables.

Real-World Analogy

Imagine you are sending a letter to Japan but you only speak English and the recipient only speaks Japanese. You need a translator. ORM is that translator between the Java object world and the SQL relational world.

Java World (ORM Handles)	Database World (SQL Tables)

Class: Product	Table: product
Object: new Product()	Row: one record in the product table
Field: private String name	Column: name VARCHAR(255)
Method: save(product)	SQL: INSERT INTO product VALUES (...)
Method: findAll()	SQL: SELECT * FROM product
Method: findById(1)	SQL: SELECT * FROM product WHERE id = 1
Method: deleteById(1)	SQL: DELETE FROM product WHERE id = 1

15. Types of ORM Frameworks

There are many ORM frameworks available across different programming languages. In the Java ecosystem, these are the most important ones:

ORM Framework	Description
Hibernate (Java)	The most popular Java ORM. Implements the JPA specification. Used under the hood by Spring Boot's Spring Data JPA. Industry standard.
Spring Data JPA (Java)	A Spring abstraction on top of JPA/Hibernate. Gives you repositories with free CRUD methods. This is what you used in the PMS project.
EclipseLink (Java)	The reference implementation of JPA. Less commonly used than Hibernate in practice but fully JPA-compliant.
MyBatis (Java)	A semi-ORM — you write SQL but MyBatis maps results to Java objects. More control, less automation than Hibernate.
Entity Framework (.NET)	Microsoft's ORM for C#/.NET applications. The Hibernate equivalent in the .NET world.
SQLAlchemy (Python)	The dominant ORM in Python. Used with Flask and FastAPI.
Django ORM (Python)	Built into the Django web framework. Very beginner-friendly.
Sequelize (Node.js)	Popular ORM for JavaScript/TypeScript with Node.js.

📌 **What You Need to Know**

In Java/Spring Boot development, Hibernate IS the ORM. When you use Spring Data JPA (like in the PMS project), Spring Boot automatically uses Hibernate as the underlying JPA implementation. You benefit from Hibernate without configuring it manually.

16. What is JDBC? — The Old Way

JDBC stands for Java Database Connectivity. It is Java's built-in API for connecting to databases and executing SQL. Before ORM frameworks like Hibernate existed, every Java developer used JDBC directly.

JDBC is a low-level API — it gives you full control but requires you to write a lot of repetitive, error-prone code for every single database operation.

What a Simple JDBC Operation Looks Like

Here is what fetching a product by ID looks like with raw JDBC:

```java
// JDBC — Fetching a product by ID (the hard way)


Connection conn = null;
PreparedStatement stmt = null;
ResultSet rs = null;
Product product = null;


try {
    // Step 1: Load the JDBC driver
    Class.forName("com.mysql.cj.jdbc.Driver");


    // Step 2: Create a connection
    conn = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/pms", "root", "password");


    // Step 3: Write SQL manually
    String sql = "SELECT * FROM product WHERE id = ?";
    stmt = conn.prepareStatement(sql);
    stmt.setLong(1, id);


    // Step 4: Execute and get ResultSet
    rs = stmt.executeQuery();


    // Step 5: Manually map each column to Java field
    if (rs.next()) {
        product = new Product();
        product.setId(rs.getLong("id"));
        product.setName(rs.getString("name"));
        product.setDescription(rs.getString("Description"));
        product.setCategory(rs.getString("Category"));
        product.setPrice(rs.getInt("price"));
        product.setStockQuantity(rs.getInt("stockQuantity"));
```

The Same Operation with Hibernate / Spring Data JPA

```
// Hibernate (JPA) — Fetching a product by ID (the easy way)


Optional<Product> optional = repo.findById(id);

return optional.isPresent() ? optional.get() : null;


// That is 2 lines. Hibernate handles EVERYTHING else automatically.
```

🤯 The Difference in Numbers

JDBC: ~35 lines to fetch one record. Hibernate/JPA: 2 lines. For a full CRUD application with 6 operations, JDBC would need 150-200 lines of boilerplate code. Hibernate reduces it to ~12 lines. That is why Hibernate exists.

17. JDBC vs Hibernate — Complete Comparison

Feature	JDBC vs Hibernate
Code Volume	JDBC: 30-50 lines per operation. Hibernate: 1-3 lines per operation.
SQL Writing	JDBC: You write every SQL query manually. Hibernate: SQL is auto-generated from Java code.
Result Mapping	JDBC: Manually map each ResultSet column to Java fields. Hibernate: Auto-maps columns to fields using @Entity annotations.
Connection Management	JDBC: Open/close connections manually in every method. Hibernate: Connection pooling managed automatically.
Transaction Handling	JDBC: Manually call commit() and rollback(). Hibernate: Use @Transactional annotation — one word handles everything.
Database Portability	JDBC: SQL queries are often database-specific (MySQL syntax differs from Oracle). Hibernate: Switch databases by changing one config line — HQL works on all databases.
Caching	JDBC: No caching — every call hits the database. Hibernate: First-level cache built-in; second-level cache available.
Learning Curve	JDBC: Easier to start (just SQL). Hibernate: Steeper initial curve but saves enormous time long-term.
Performance Control	JDBC: Full control — you write exactly what runs. Hibernate: Less control but optimizations available through configuration.
Lazy Loading	JDBC: Not available — you fetch everything manually. Hibernate: Supports lazy loading (fetch related data only when needed).
When to Use	JDBC: Legacy systems, fine-grained SQL control, batch operations. Hibernate: New projects, rapid development, standard CRUD applications.

✅ **The Industry Verdict**

For 95% of enterprise Java applications, Hibernate (via JPA/Spring Data) is the right choice. It eliminates boilerplate, reduces bugs, and speeds up development dramatically. JDBC is still useful for complex batch operations or cases where you need exact query control, but it is rarely your first choice.

18. Core Hibernate Concepts — Everything You Need to Know

18.1 What is Hibernate?

Hibernate is an open-source ORM framework for Java. It was created by Gavin King in 2001 and is now maintained by Red Hat. Hibernate implements the JPA (Java Persistence API) specification — meaning if you learn Hibernate, you know JPA.

Hibernate sits between your Java application and your database. It translates Java object operations into SQL and handles the entire database communication lifecycle.

Hibernate Concept	What It Means
JPA (Java Persistence API)	A Java specification (a standard/contract) that defines how ORM should work. Hibernate is the most popular implementation of JPA.
SessionFactory	A heavyweight, thread-safe factory object created once per application. It creates Session objects. Expensive to create — only one per database.
Session	A lightweight, single-threaded unit of work. Wraps a JDBC connection. Used to perform CRUD operations. Created from SessionFactory.
Transaction	A unit of work that is either fully completed or fully rolled back. In Hibernate, wrap operations in beginTransaction() and commit().
Persistent Object	A Java object whose state is managed by Hibernate and synchronized with the database.
First-Level Cache	Session-level cache. Within one Session, the same object is not fetched from DB twice. Automatic and always active.
Second-Level Cache	SessionFactory-level cache. Shared across Sessions. Optional — must be configured with providers like EhCache.
Lazy Loading	Related entities are not fetched until they are accessed. Prevents unnecessary database queries.
Eager Loading	Related entities are fetched immediately along with the parent entity. Use when you always need the related data.

Cascade	Operations on a parent entity automatically apply to child entities (e.g., save parent = save children too).
DDL Auto	Hibernate can auto-create or update database tables based on your entity classes.

18.2 Entity States in Hibernate

Every Hibernate entity (Java object) exists in one of four states. Understanding these states prevents many common bugs:

State	Description & Example
Transient	Object is created with 'new' but NOT saved to DB yet. Hibernate does NOT track it. Example: Product p = new Product(); (p is transient — not in DB)
Persistent	Object is associated with a Hibernate Session AND synchronized with DB. Changes to it are auto-detected. Example: After session.save(p) or repo.findById(id).get()
Detached	Object was persistent but its Session was closed. Changes are NOT tracked. Example: After the transaction ends in a REST API call — the returned Product is detached.
Removed	Object is scheduled for deletion from DB. Will be deleted when transaction commits. Example: After session.delete(p) or repo.deleteById(id).

🧠 **State Memory Trick**

Think of it like an employee's relationship with HR: Transient = Not hired yet (no records). Persistent = Currently employed (HR tracks everything). Detached = On leave (was employed, now temporarily not tracked). Removed = Resigned (will be removed from all records).

18.3 Hibernate Annotations

Annotation	Purpose
@Entity	Marks class as a DB table. Required for Hibernate to manage it.
@Table(name="product")	Specifies the table name. If omitted, table name = class name (lowercase).
@Id	Marks primary key field. Every entity must have one.
@GeneratedValue(strategy=...)	AUTO: Hibernate decides. IDENTITY: DB auto-increment. SEQUENCE: DB sequence. TABLE: Hibernate manages a counter table.
@Column(name="prod_name", nullable=false, length=100)	Maps field to specific column with constraints.
@Transient	Field is NOT persisted to DB. Hibernate ignores it.
@OneToOne	One entity relates to exactly one of another (e.g., Person → Passport).
@OneToMany	One entity relates to many of another (e.g., Category → Products).
@ManyToOne	Many entities relate to one (e.g., Products → Category).
@ManyToMany	Many entities relate to many (e.g., Students → Courses).
@JoinColumn(name="category_id")	Specifies the foreign key column for relationships.
@Lob	Maps large objects (BLOB/CLOB) — for storing images or long text in DB.
@Temporal(TemporalType.DATE)	Maps Java Date fields to SQL DATE, TIME, or TIMESTAMP.
@Enumerated(EnumType.STRING)	Maps Java enum fields to DB as string (or ordinal).

19. hibernate.cfg.xml — How It Works

In traditional Hibernate (without Spring Boot), all configuration is done in a file called hibernate.cfg.xml. This file tells Hibernate EVERYTHING it needs to know: which database to connect to, what credentials to use, how to behave, and which entity classes to manage.

In Spring Boot, this configuration moves to application.properties or application.yml — but understanding hibernate.cfg.xml is essential because it reveals what is happening under the hood even in Spring Boot.

Full hibernate.cfg.xml with Explanation

```xml
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
    <session-factory>

        <!-- ===== DATABASE CONNECTION SETTINGS ===== -->

        <!-- JDBC Driver class for your database -->
        <property name="hibernate.connection.driver_class">
            com.mysql.cj.jdbc.Driver
        </property>

        <!-- Database URL -->
        <!-- Format: jdbc:mysql://host:port/database_name -->
        <property name="hibernate.connection.url">
            jdbc:mysql://localhost:3306/pms_db
        </property>

        <!-- Database username -->
        <property name="hibernate.connection.username">root</property>

        <!-- Database password -->
        <property name="hibernate.connection.password">yourpassword</property>

        <!-- ===== HIBERNATE BEHAVIOR SETTINGS ===== -->

        <!-- Database dialect: tells Hibernate which SQL flavor to generate -->
        <!-- Use MySQL8Dialect for MySQL 8+, PostgreSQLDialect for PostgreSQL -->
        <property name="hibernate.dialect">
```

Equivalent Spring Boot application.properties

In Spring Boot, you never need hibernate.cfg.xml. The same settings go in src/main/resources/application.properties:

```
# Database connection
spring.datasource.url=jdbc:mysql://localhost:3306/pms_db
spring.datasource.username=root
spring.datasource.password=yourpassword
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Hibernate / JPA settings
spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# DDL auto: create / create-drop / update / validate / none
spring.jpa.hibernate.ddl-auto=update
```

DDL-Auto Value	What Happens (When to Use)
create	Drops ALL tables and recreates them every time the app starts. ALL DATA IS LOST. Use only in early development.
create-drop	Creates tables on start, drops them on shutdown. Perfect for automated testing.
update	Adds new columns/tables if entities change. Never drops existing columns. Safe for development.
validate	Compares entity classes to the actual DB schema. Throws error if they don't match. Use in production.
none	Hibernate does nothing with the schema. You manage DB manually. Use when DBAs control the database.

20. HQL — Hibernate Query Language

HQL (Hibernate Query Language) is Hibernate's own query language. It looks exactly like SQL but with one critical difference: HQL works with Java class names and field names, not database table names and column names.

🔑 **The Golden Rule of HQL**

In HQL, you ALWAYS use Java class names and Java field names — NEVER the database table name or column name. HQL: 'FROM Product WHERE name = :name'. SQL: 'SELECT * FROM product WHERE name = ?'. Hibernate translates HQL to proper SQL automatically.

HQL vs SQL — Side by Side

SQL (Database Language)	HQL (Hibernate Language)
SELECT * FROM product	FROM Product (or: SELECT p FROM Product p)
SELECT * FROM product WHERE id = 1	FROM Product WHERE id = 1
SELECT * FROM product WHERE name = 'Laptop'	FROM Product WHERE name = 'Laptop'
SELECT * FROM product WHERE inStock = true	FROM Product WHERE inStock = true
SELECT * FROM product ORDER BY price DESC	FROM Product ORDER BY price DESC
SELECT COUNT(*) FROM product	SELECT COUNT(p) FROM Product p
DELETE FROM product WHERE id = 1	DELETE FROM Product WHERE id = 1
UPDATE product SET price=5000 WHERE id=1	UPDATE Product SET price = 5000 WHERE id = 1

HQL Syntax Basics

```
// Basic select all
FROM Product

// With alias (recommended style)
SELECT p FROM Product p

// WHERE clause
FROM Product WHERE name = 'Laptop'

// Named parameter (safer than string concatenation)
FROM Product WHERE name = :productName

// Positional parameter
FROM Product WHERE name = ?1

// Multiple conditions
FROM Product WHERE price > 1000 AND inStock = true

// ORDER BY
FROM Product ORDER BY price DESC

// LIKE (case sensitive in HQL)
FROM Product WHERE name LIKE '%Laptop%'

// Aggregate functions
SELECT COUNT(p) FROM Product p
SELECT MIN(p.price) FROM Product p
SELECT MAX(p.price) FROM Product p
SELECT AVG(p.price) FROM Product p

// Specific fields (returns Object[], not Product)
SELECT p.name, p.price FROM Product p
```

21. HCQL — Hibernate Criteria Query Language

HCQL (Hibernate Criteria Query Language), also called the Criteria API, is a programmatic way to build queries using Java code instead of writing query strings. This makes queries type-safe and refactor-friendly.

The advantage of Criteria API over HQL: if you rename a Java field, the compiler immediately catches broken queries. With HQL strings, the bug only appears at runtime.

HQL vs Criteria API — Same Query, Different Style

HQL String Approach	Criteria API Approach
String hql = "FROM Product WHERE price > :price"; Query q = session.createQuery(hql); q.setParameter("price", 1000);	CriteriaBuilder cb = session.getCriteriaBuilder(); CriteriaQuery<Product> cq = cb.createQuery(Product.class); Root<Product> root = cq.from(Product.class); cq.select(root).where(cb.gt(root.get("price"), 1000));

Criteria API Example in Code

```
// Using JPA Criteria API with EntityManager

@Autowired
EntityManager entityManager;

public List<Product> findByPriceGreaterThan(int minPrice) {
    CriteriaBuilder cb = entityManager.getCriteriaBuilder();
    CriteriaQuery<Product> cq = cb.createQuery(Product.class);
    Root<Product> root = cq.from(Product.class);

    // WHERE price > minPrice
    Predicate pricePredicate = cb.gt(root.get("price"), minPrice);
    cq.select(root).where(pricePredicate);

    // ORDER BY price ASC
    cq.orderBy(cb.asc(root.get("price")));

    return entityManager.createQuery(cq).getResultList();
}

public List<Product> findInStockByCategory(String category) {
    CriteriaBuilder cb = entityManager.getCriteriaBuilder();
    CriteriaQuery<Product> cq = cb.createQuery(Product.class);
    Root<Product> root = cq.from(Product.class);

    // WHERE category = ? AND inStock = true
    Predicate catPred = cb.equal(root.get("Category"), category);
    Predicate stockPred = cb.isTrue(root.get("inStock"));
    cq.select(root).where(cb.and(catPred, stockPred));

    return entityManager.createQuery(cq).getResultList();
}
```

💡 When to Use Criteria API vs HQL

Use HQL for simple, fixed queries — it is cleaner and easier to read. Use Criteria API for dynamic queries where conditions are built at runtime (e.g., a search filter where the user may or may not fill in each field). In Spring Boot, Spring Data JPA's Specification interface is an elegant way to use the Criteria API.

22. Native SQL in Hibernate

Native SQL means writing plain, raw SQL queries exactly as you would in a database tool like MySQL Workbench or pgAdmin. Hibernate allows this when HQL is not enough — for example, when using database-specific functions, complex joins, or stored procedures.

Query Type	When to Use
HQL	Standard CRUD, simple filters, sorting, pagination — works across all databases.
Criteria API	Dynamic queries where conditions are built programmatically at runtime.
Native SQL	Complex database-specific queries, stored procedure calls, raw performance optimization, or queries HQL cannot express.

In Spring Data JPA, native SQL is used via the @Query annotation on Repository methods with nativeQuery = true.

23. HQL & Native SQL — 10 Real Examples in ProductRepository

The following examples show how to add custom queries directly in the Repository interface using Spring Data JPA's @Query annotation. These examples use the Product entity from the PMS project.

> 📌 **Two Modes of @Query**
>
> Without nativeQuery=true: HQL mode — use Java class name (Product) and Java field names. With nativeQuery=true: Native SQL mode — use DB table name (product) and column names. When in doubt, prefer HQL unless you need database-specific features.

The Complete ProductRepository with 10 Custom Queries

```java
package com.pms.repository;


import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import com.pms.model.Product;


@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {


    // ────────────────────────────────────────────────────────────
    // EXAMPLE 1: Find by Name (HQL)
    // Use: Find a product whose name exactly matches
    // When: User searches for an exact product name
    // ────────────────────────────────────────────────────────────
    @Query("FROM Product WHERE name = :name")
    List<Product> findByName(@Param("name") String name);



    // ────────────────────────────────────────────────────────────
    // EXAMPLE 2: Find by Name Containing (HQL) — Search/Like
    // Use: Search products whose name contains a keyword
    // When: User types 'lap' and you want 'Laptop', 'LapPad' etc.
    // ────────────────────────────────────────────────────────────
    @Query("FROM Product p WHERE p.name LIKE %:keyword%")
    List<Product> findByNameContaining(@Param("keyword") String keyword);



    // ────────────────────────────────────────────────────────────
    // EXAMPLE 3: Find by inStock Status (HQL)
```

How to Use These Methods — Service & DAO Examples

After adding these methods to the Repository, call them from the DAO just like the built-in methods:

```java
// In ProductDao.java — using the custom repository methods


// Get all products by category
public List<Product> getByCategory(String category) {
    return repo.findByCategory(category);
}


// Search products by keyword
public List<Product> searchByName(String keyword) {
    return repo.findByNameContaining(keyword);
}


// Get only in-stock products
public List<Product> getInStockProducts() {
    return repo.findByInStock(true);
}


// Get products under a price (budget filter)
public List<Product> getBudgetProducts(int maxPrice) {
    return repo.findByPriceLessThan(maxPrice);
}


// Get products in price range
public List<Product> getByPriceRange(int min, int max) {
    return repo.findByPriceRange(min, max);
}
```

Quick Reference: HQL vs Native SQL for These Examples

Example	HQL or Native
findByName	HQL — Simple equality filter, database-agnostic
findByNameContaining	HQL — LIKE with wildcards, works on all databases
findByInStock	HQL — Boolean field filter
findByCategory	HQL — String equality with alias syntax
findByPriceLessThan	HQL — Comparison with ORDER BY
findInStockByCategory	HQL — Two conditions with AND
countByCategory	HQL — Aggregate COUNT function
findByNameNative	Native SQL — Direct table/column access
findByStockStatusNative	Native SQL — DB-level boolean (0/1 in MySQL)
findByPriceRange	Native SQL — BETWEEN for range queries

🚀 Spring Data JPA Bonus: Derived Query Methods

You can skip @Query entirely for simple cases! Spring Data JPA can generate queries from method names: findByName(String name), findByInStock(boolean status), findByCategoryAndInStock(String cat, boolean stock), findByPriceLessThan(int price). Spring reads the method name and builds the query automatically. Use @Query only when the method name would become too long or complex.

24. Hibernate Complete Summary — Never Forget

Concept	One-Line Memory Hook
ORM	Translator between Java objects and SQL tables — one class = one table
JDBC	Manual, verbose, error-prone — 35 lines for one query. The old way.
Hibernate	ORM implementation for Java — replaces JDBC boilerplate with annotations
JPA	The Java standard/spec that Hibernate implements
Spring Data JPA	Spring's abstraction on top of JPA — gives you free CRUD via JpaRepository
hibernate.cfg.xml	Hibernate's config file — DB connection, dialect, ddl-auto, entity mappings
SessionFactory	One per app — creates Sessions — heavyweight, thread-safe
Session	One per request/transaction — used to do actual DB operations
HQL	SQL but with Java class/field names instead of table/column names
Criteria API	Build queries in Java code — type-safe, great for dynamic filters
Native SQL	@Query(nativeQuery=true) — raw SQL when HQL is not enough
@Param	Binds method parameter to named parameter in @Query string
ddl-auto=update	Hibernate auto-updates schema — safe for dev, validate for prod
Lazy Loading	Don't fetch related data until you access it — saves DB calls
First-Level Cache	Same object not fetched twice in one Session — automatic

— End of Hibernate Chapter —